

Съдържание

1	Стъпка 03 - Варианти на CFR и методи на Монте Карло: Доклад за реализацията	1
1.1	1. Разработени елементи	1
1.2	2. Игрови двигател за Ледюк покер	2
1.3	3. Реализации на алгоритми	2
1.4	4. Фаза на изследване (референция OpenSpiel)	4
1.5	5. Еталонен тест с ограничение във времето - 180 секунди реално време	7
1.6	6. Кръстосана проверка спрямо OpenSpiel	8
1.7	7. Възпроизвеждане	8

1 Стъпка 03 - Варианти на CFR и методи на Монте Карло: Доклад за реализацията

Среда: април 2026 г.

Игра: Ледюк покер (6 карти, 2 играчи, 2 рунда на залагане)

Алгоритми: обикновен вариант на CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling

Цели: експлоатируемост на CFR+ < 0.001 в рамките на ≤ 180 s · кръстосана проверка спрямо OpenSpiel · наклон в лог-лог мащаб ≈ -0.5 за MCCFR

Статус: Всички цели са постигнати ✓

1.1 1. Разработени елементи

Две направления на работа: **фаза на изследване**, използваща референтните решаващи програми на OpenSpiel за изграждане на интуиция и установяване на опорни базови резултати, последвана от **реализация от нулата** на всичките четири алгоритъма, ръчно кодирани единствено в Python + NumPy.

Структура на кода:

implementation/step03/

```
|— cfr/
| |— leduc_poker.py          # Full game engine (6 cards, 2 rounds, community card)
| |— info_set_node.py       # Regret & strategy storage per info set
| |— cfr_trainer.py         # Vanilla CFR (full-traversal, buffered regrets)
| |— cfrplus_trainer.py     # CFR+ (flooring + linear avg + alternating)
| |— mccfr_external_trainer.py # External Sampling MCCFR
| |— mccfr_outcome_trainer.py # Outcome Sampling MCCFR ( $\epsilon$ -on-policy + IS)
| |— train.py               # Single-algorithm training entry point
```

```

|   └─ train_all_timed.py           # 180 s wall-clock benchmark harness
└─ evaluate/
|   └─ best_response.py             # Info-set-constrained best response
|   └─ exploitability.py           # BR0 + BR1 exact exploitability
|   └─ convergence.py              # Geometric-spaced snapshot logger
└─ exploration/
|   └─ implDayOne1.py               # OpenSpiel five-algorithm comparison on Kuhn
|   └─ leduc_comparison.py          # OpenSpiel four-algorithm comparison on Leduc
|   └─ leduc_race.py                # 5-minute wall-clock race
└─ compare_openspiel.py            # Cross-validation against OpenSpiel solvers
└─ utils/plotting.py               # Figure generation

```

1.2 2. Игрови двигател за Ледюк покер

Пълна реализация, съответстваща на семантиката на OpenSpiel: шест карти ($\{J, Q, K\} \times 2$ бои), два кръга на залагане с разкрита обща карта между тях, действия пас/зalog/повишаване, фиксирани размери на залога (2 в първия кръг, 4 във втория кръг), максимум две повишавания в един кръг. Класиране на ръка по двойки: тайна карта, която съвпада с общата карта, побеждава всяка висока карта. Изброява всички 120 пермутации на раздаването за точно изчисляване на математическо очакване.

1.3 3. Реализации на алгоритми

1.3.1 3.1 Обикновен вариант на CFR

Пълно обхождане на дърво с вероятностна извадка върху всичките 120 раздавания при всяка итерация. Актуализациите на съжалението се буферират за всяко информационно множество и се прилагат атомарно в края на обхождането на всяко раздаване, с цел да се избегне отклонение от стратегията по средата на итерацията.

1.3.2 3.2 CFR+

Три модификации на обикновения вариант на CFR, всяка от които е локализирана. Основната промяна е стъпката за подово ограничаване на съжалението:

```

# cfrplus_trainer.py - regret flooring (ReLU-style clip)
for info_set, deltas in regret_buffer.items():
    node = node_map[info_set]

```

```

for a in range(node.num_actions):
    node.regret_sum[a] = max(node.regret_sum[a] + deltas[a], 0.0)

```

Усредняването на линейни стратегии претегля итерация t със стойността на самата t в плъзгащата средна; редуващите се актуализации напредват само със съжаленията на един играч при всяко преминаване (играч 0 при нечетните итерации, играч 1 при четните).

1.3.3 3.3 Външно вземане на проби в MCCFR

Във възловите състояния на traverser-а се изследват всички действия, докато във възловите състояния на противника се взема проба само от едно действие, избрано според текущата стратегия на противника.

```

def external_cfr(state, update_player):
    if state.is_terminal():
        return state.get_utility(update_player)

    player = state.current_player()
    strategy = get_strategy(state.info_set(player))

    if player == update_player:
        # explore ALL actions (same as vanilla CFR)
        values = [external_cfr(state.apply(a), update_player)
                   for a in legal_actions]
        node_value = sum(strategy[a] * values[a] for a in legal_actions)
        for a in legal_actions:
            regret[info_set][a] += values[a] - node_value
        return node_value
    else:
        # SAMPLE one opponent action from the current strategy
        sampled_action = sample(strategy)
        return external_cfr(state.apply(sampled_action), update_player)

```

Разходът на итерация е приблизително 42 възела (срещу 20,400 при пълно обхождане).

1.3.4 3.4 MCCFR Outcome Sampling

Изважда една единствена корен-краен траектория. Във възлите на **traverser** действията се изтеглят от ϵ -on-policy mixture ($\epsilon = 0.6$: равномерно с вероятност ϵ , в противен случай текущата стратегия). Актуализациите на съжалението се коригират чрез извадка по важност - отношението между истинската вероятност за достигане и вероятността за извадка - като

по този начин се запазва свойството на безпристрастен оценител. Разходът на итерация е приблизително 5.5 възела.

1.3.5 3.5 Оценител на експлоатируемостта

Итеративен най-добър отговор, ограничен до информационно множество: за всеки играч се изчислява оптималната контрастратегия при условие, че отговарящият трябва да изиграе едно и също действие във всички състояния в рамките на дадено информационно множество. Връща $BR_0(\sigma_1) + BR_1(\sigma_0)$ като точна експлоатируемост.

1.4 4. Фаза на изследване (референция OpenSpiel)

1.4.1 4.1 Кун покер - 5000 итерации

За проверка за коректност в малък **мащаб** бяха стартирани и четирите алгоритъма на OpenSpiel, както и персонализираният CFR от Стъпка 02.

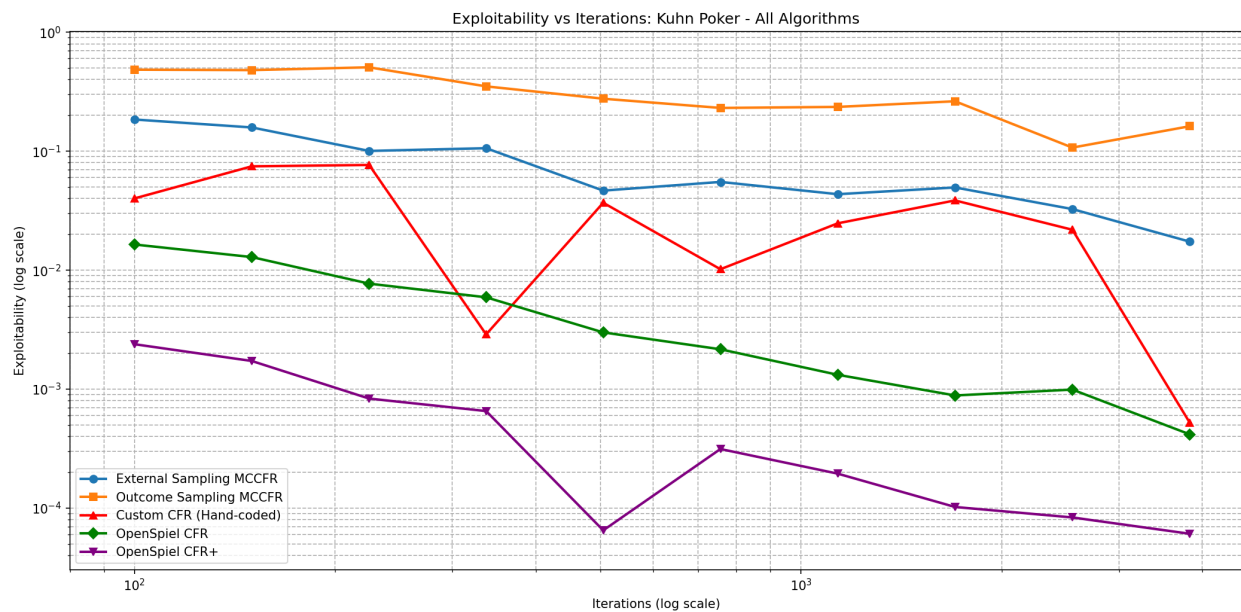
Алгоритъм	Експлоатируемост	Време
Персонализиран CFR (Стъпка 02)	$\sim 3.5 \times 10^{-4}$	< 1 s
OpenSpiel CFR	$\sim 1.5 \times 10^{-3}$	~2 s
CFR+	$\sim 3.0 \times 10^{-4}$	~2 s
MCCFR с външно вземане на проби	$\sim 4 \times 10^{-3}$	~1 s
MCCFR с извадка по резултати	$\sim 2.5 \times 10^{-2}$	~1 s

Всички алгоритми достигат почти равновесие на Наш за секунди; разликата е само от академично значение при този мащаб.

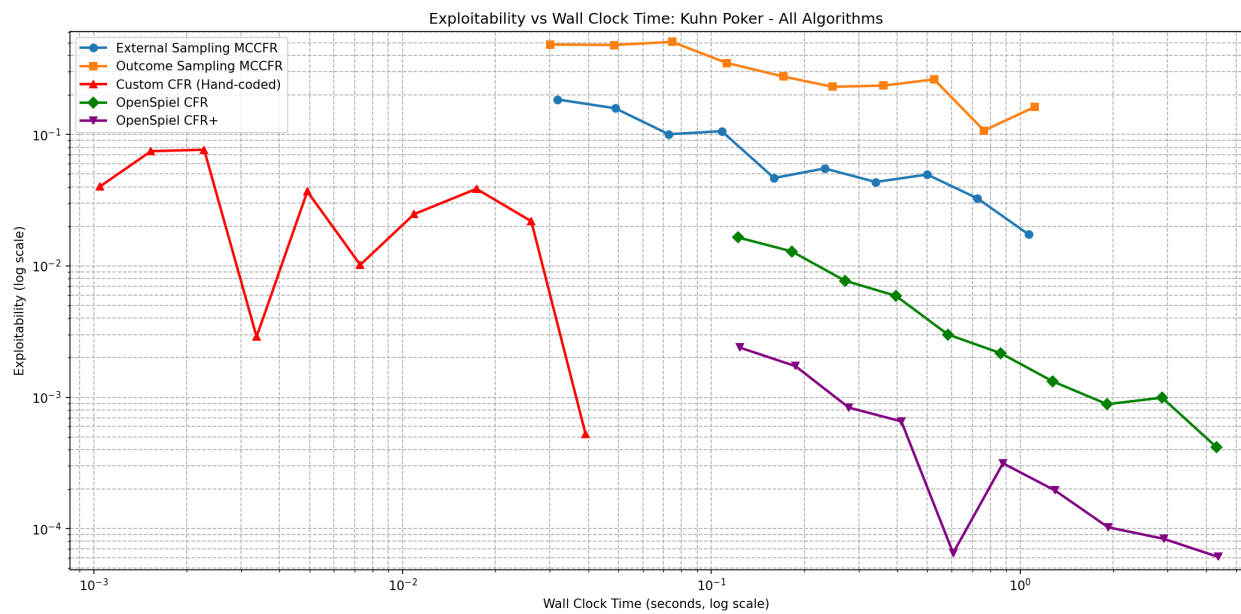
1.4.2 4.2 Ледюк покер - 5,000 итерации

Алгоритъм	Експлоатируемост при 5k итерации	Време
CFR+	$\sim 5.4 \times 10^{-5}$	~859 s
Обикновен вариант на CFR	$\sim 7.6 \times 10^{-3}$	~747 s
MCCFR с външно вземане на проби	~1.17	~7 s
MCCFR с извадка по резултати	~3.08	~5 s

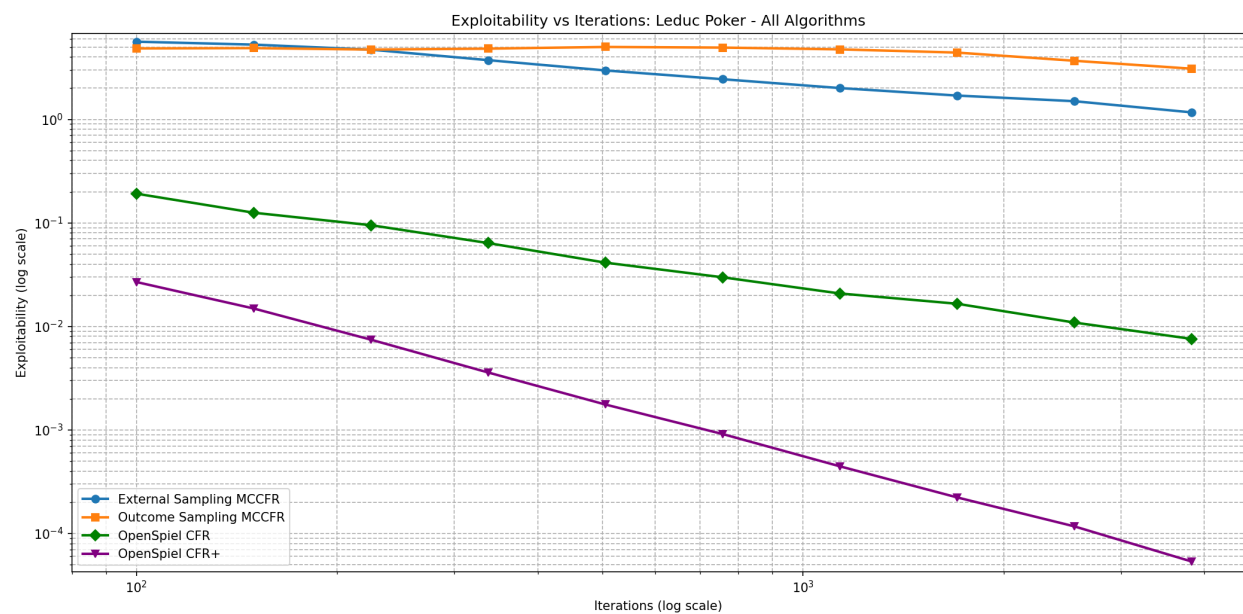
Методите с пълно обхождане доминират с 4–5 порядъка на итерация; измерени в реално време, разликата намалява, но остава при дървета с размер, съпоставим с този на Ледюк.



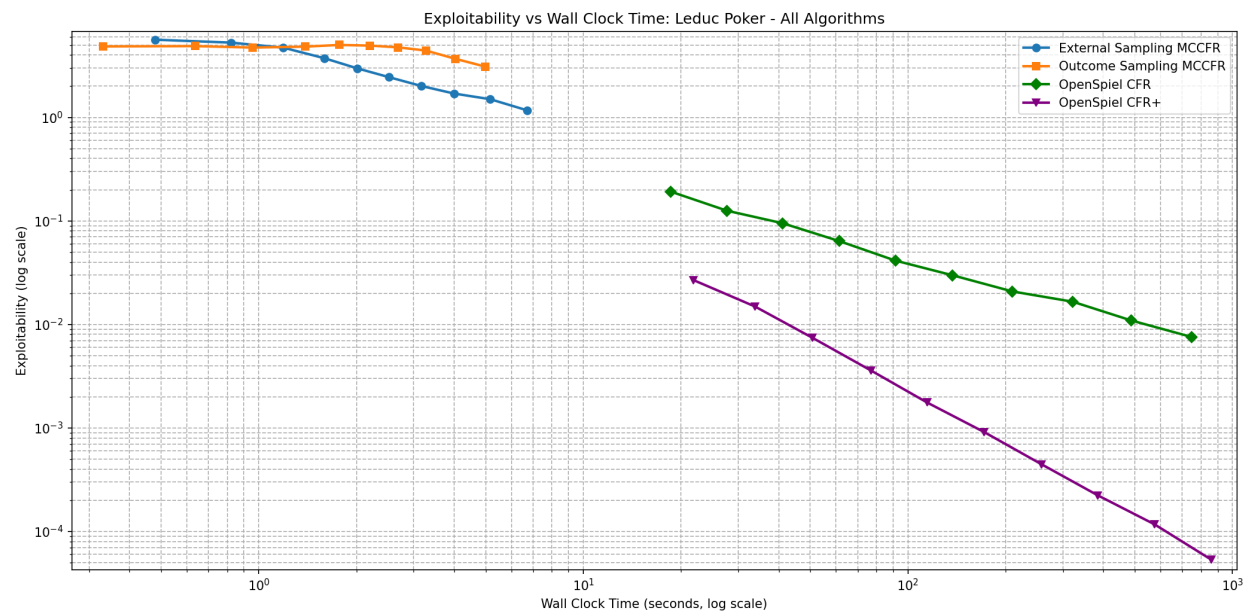
Фигура 1: Kuhn - Exploitability vs Iterations



Фигура 2: Kuhn - Exploitability vs Wall-Clock Time



Фигура 3: Ледюк - Exploitability vs Iterations

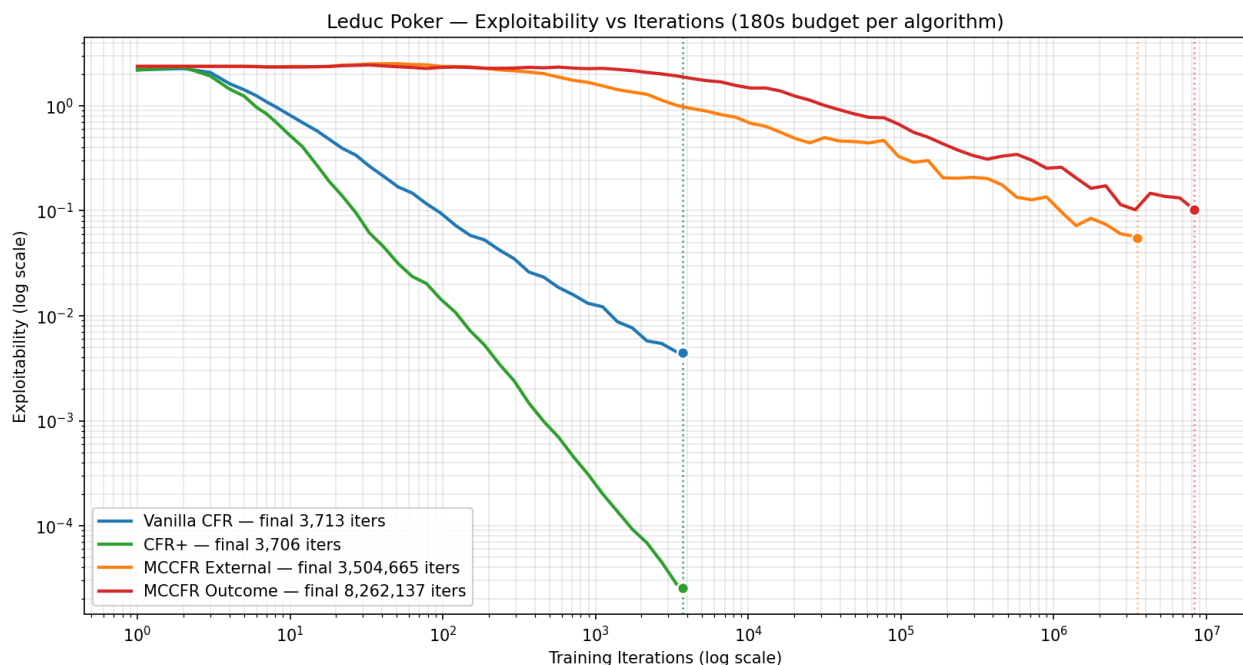


Фигура 4: Ледюк - Exploitability vs Wall-Clock Time

1.5 5. Еталонен тест с ограничение във времето - 180 секунди реално време

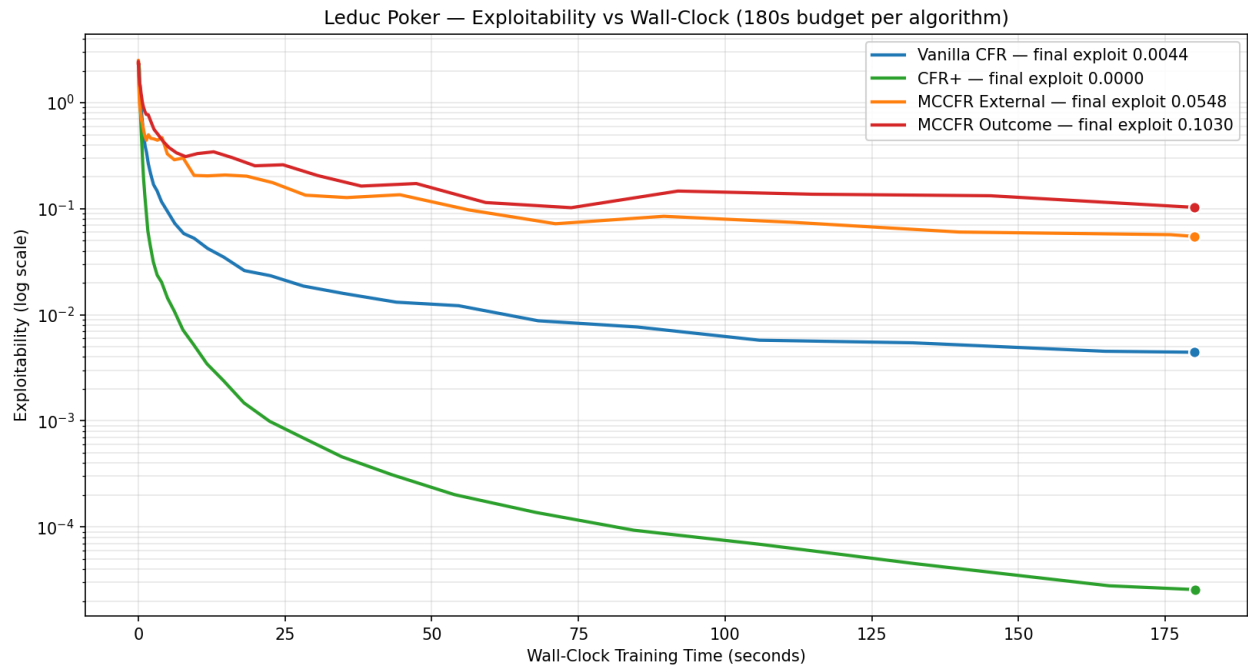
`train_all_timed.py` стартира всяка персонализирана имплементация при общ бюджет от 180 секунди, като използва геометрично разположени моментни снимки. Това представлява справедливото сравнение за компромиса между дисперсия и скорост.

Алгоритъм	Итерации	Крайна експлоатируемост	Информационни множества
Обикновен вариант на CFR	3,713	4.4×10^{-3}	936
CFR+	3,706	2.6×10^{-5}	936
MCCFR External	3,504,665	5.5×10^{-2}	936
MCCFR Outcome	8,262,137	1.0×10^{-1}	936



Фигура 5: Exploitability vs Iterations (log-log)

CFR+ достига почти точното **наш равновесие** (2.6×10^{-5}) за 3 минути - над 150 пъти по-добър резултат от обикновения вариант на CFR, въпреки че извършва само малко по-малко **ите-рации**. И двата варианта на MCCFR, въпреки милионите **ите-рации**, остават с 3–4 порядъка по-лоши при този **размер на играта**, както е предвидено от анализа на **дисперсията** и **ско-ростта** в резюмето.



Фигура 6: Exploitability vs Wall-Clock Time (log-y)

1.6 6. Кръстосана проверка спрямо OpenSpiel

И двете реализации с пълно **обхождане** се валидират спрямо референтните решаващи алгоритми на OpenSpiel при еднакъв **брой итерации** - 500.

Алгоритъм	Наша реализация (500 итерации)	OpenSpiel (500 итерации)	Съвпадение
Обикновен вариант на CFR	0.020	0.022	✓
CFR+	8.6×10^{-4}	9.4×10^{-4}	✓

Малки разлики се наблюдават поради подреждането на сделките и случайните начални стойности; и двете сходяват към едно и също **равновесие на Неш** в рамките на допустимия шум.

1.7 7. Възпроизвеждане

From repo root, with .venv activated:

Train a single algorithm (configurable in train.py):

`python implementation/step03/cfr/train.py`

Run the 180 s timed benchmark for all four algorithms:

`python implementation/step03/cfr/train_all_timed.py`

Exploration - OpenSpiel reference comparisons:

`python implementation/step03/exploration/implDayOne1.py` *# Kuhn*

`python implementation/step03/exploration/leduc_comparison.py` *# Leduc, iteration budget*

`python implementation/step03/exploration/leduc_race.py` *# Leduc, 5-min wall-clock*

Cross-validate custom vs OpenSpiel:

`python implementation/step03/compare_openspiel.py`

Генерираните фигури са налични в deliverables/reports/step03/figures/.